The Plain Spec Governor Skill for OpenCLAW and OmegaCLAW

Draft design note for human review

July 2026

Abstract

This document explains the purpose and design of the OpenCLAW skill file `openclaw_plain_spec_governor_skill`. The skill introduces a Plain-before-code discipline for research code, command-line scripts, and agent codebase revisions. Its central mechanism is a persistent reviewer subagent, `PlainSpecReviewer`, which reviews Plain specifications before any implementation code is written. The intent is not to replace future formal verification, but to create an intermediate layer of LLM-assisted specification reasoning that can catch ambiguity, missing tests, unsafe operational behavior, misleading research design, and brittle architecture before these defects become code.

Contents

1 Purpose of the Skill

The skill file exists to make a coding agent pause before implementation and ask: what exactly is the code supposed to do, how will we know whether it did it, and what mistakes might a plausible implementation make while still seeming to satisfy the request?

The target use case is OpenCLAW/OmegaCLAW research and operations work, including:

- Python research code for symbolic or subsymbolic AI experiments;
- MeTTa, MM2, or related symbolic-language experiments;
- Linux command-line scripts for environment setup or agent operation;
- revisions to OpenCLAW/OmegaCLAW agent codebases;
- future UX/UI code where behavior matters.

The immediate motivation is pragmatic. Formal verification may eventually be used on Plain specifications or on artifacts rendered from them. But long before the formal pipeline is mature, a significant fraction of failures can be found by asking a strong LLM to reason adversarially about the spec. The skill therefore creates a governed pre-code review loop: first write or update a Plain spec, then have a persistent reviewer inspect it, then revise the spec, and only then allow implementation.

The skill is intentionally not a heavyweight process for every trivial edit. It contains a light path for small scripts and a more intensive path for research code and architectural changes. The core idea is to match review cost to risk.

2 Why Plain Is the Right Boundary

Plain is useful here because it gives the agent a structured, textual requirements artifact rather than a loose chat instruction. The Plain language guide describes Plain as a specification language for writing software requirements in a clear, structured format. Its standard sections include `***definitions***`, `***implementation reqs***`, `***test reqs***`, and `***functional specs***`. It also supports nested `***acceptance tests***` under functional specs. See the Plain documentation at https://plainlang.org/docs/language-guide/basic-syntax/.

For an agentic coding workflow, this matters for several reasons.

First, Plain separates roles that are often blurred in chat-based coding. The `definitions` section records the domain vocabulary. The `implementation reqs` section says how the code should be constrained or structured. The `test reqs` section says how conformance tests should be built or executed. The `functional specs` section says what behavior the software must implement. This separation gives both the coding agent and the reviewer subagent better handles for detecting errors.

Second, Plain's `:Concept:` notation makes ambiguous terms inspectable. A Plain reviewer can check whether a concept has been defined, whether it is used before definition, whether near-duplicates exist, and whether the same concept appears to drift in meaning.

Third, Plain functional specs are expected to be small and testable. The Plain docs advise that each functional-spec bullet should describe a single piece of functionality, use defined concepts

consistently, and remain clear, testable, and limited in complexity. See https://plainlang.org/docs/language-guide/functional-specs/. This fits well with LLM review: the reviewer can attack each bullet for ambiguity, edge cases, and missing acceptance tests.

Fourth, Plain source is plain text. It can be committed with code, diffed, reviewed, patched, and used as an implementation handoff artifact. This makes it suitable not only for future formal verification, but also for current human and multi-agent workflows.

3 The Main Design Principle: A Specification Gate

The skill treats the Plain spec as a pre-code contract. The coding agent should not write implementation code until the relevant Plain spec has passed a review gate or has received an explicit human waiver.

The gate is not meant to prove the spec correct. It is meant to reduce avoidable mistakes before they become more expensive. The reviewer asks questions such as:

- Are the main concepts defined and used consistently?
- Are the functional requirements small, observable, and testable?
- Are implementation constraints separated from behavior?
- Could a coding model implement the wrong thing while appearing compliant?
- Are edge cases and failure modes specified?
- For scripts, are side effects, idempotence, privilege assumptions, and partial failures specified?
- For research code, are known-answer tests, metrics, baselines, seeds, and artifact-retention rules specified?
- For agent code, are state changes, external actions, safety boundaries, and regression cases specified?

The result is a verdict: `GO`, `REVISE`, or `STOP`. A `GO` verdict permits implementation. A `REVISE` verdict requires spec changes before implementation, unless the remaining issues are explicitly accepted as non-blocking. A `STOP` verdict blocks implementation until the underlying problem is resolved or escalated.

4 Why Use a Persistent Reviewer Subagent?

A one-off review prompt is useful, but a persistent subagent is more useful for a long-lived research environment. The skill therefore creates or activates a subagent named `PlainSpecReviewer`.

The reviewer is persistent so that it can remember recurring project-specific patterns, including:

- reusable Plain concept definitions;
- naming conventions for OpenCLAW/OmegaCLAW specs;
- prior ambiguity patterns;
- common Linux setup hazards;
- recurring research-validity failures;
- preferred frameworks and templates;
- unresolved assumptions from prior reviews;
- past cases where a coding model implemented the wrong thing.

This persistence turns the reviewer from a generic critic into a project memory component. Over time, it should develop useful local taste: which concepts are usually meant by `:AgentState:`, what counts as an `:Artifact:` in a particular lab workflow, which setup-script files are dangerous to overwrite, and which research methods have previously needed synthetic sanity cases.

The reviewer subagent is not allowed to write implementation code. This restriction is important. Its job is to preserve the distinction between specification reasoning and implementation. It may propose Plain patches, acceptance tests, invariants, assumptions, and implementation gates, but it should not start solving the programming task directly.

5 Risk Tiers

The skill uses risk tiers because not all coding tasks deserve the same amount of ceremony. The following table summarizes the intended tiers.

Tier	**Typical task**	**Main risks**	**Review path**
Tier 0	Tiny local helper, log parser, config stub, one-off converter	Local logic errors, missing edge cases, unclear input/output	Light review
Tier 1	Linux setup script, command-line operational script, shell config change	Destructive side effects, privilege assumptions, non-idempotence, partial failure	Light review plus ops-safety checks
Tier 2	Agent codebase revision, persistence or orchestration change, new module/API	State corruption, brittle interfaces, regression, unsafe external action	Intensive review
Tier 3	Long-running research code, estimator, miner, benchmark, symbolic/subsymbolic experiment	Misleading results, broken tools, poor reproducibility, wrong metrics, hidden assumptions	Intensive review, sometimes two-reviewer dialectic

When uncertain, the agent should choose the higher tier if the task could waste significant compute or human time, mutate important state, or produce misleading conclusions.

6 Required Artifacts

For nontrivial tasks, the skill asks the agent to create or update three files:

```
<task-or-module>.plain
<task-or-module>.spec-review.md
<task-or-module>.assumptions.md
```

The Plain file is the source of truth. The review file records critique, verdicts, decisions, and unresolved issues. The assumptions file records assumptions that have not yet been promoted into firm requirements.

For very small tasks, the review and assumptions can be embedded in a short work note, but the agent should still create or update a minimal Plain spec unless the supervising user explicitly prohibits that.

7 The Workflow

The skill's workflow has eight steps.

7.1 Step 1: Detect Activation

The skill activates when the agent is about to write or modify code, create a script, start or pivot a research project, or change agent behavior. It also activates when implementation reveals that the current spec is incomplete or wrong.

7.2 Step 2: Classify Risk

The agent classifies the task as Tier 0, Tier 1, Tier 2, or Tier 3. This choice selects the light or intensive review path.

7.3 Step 3: Create or Update the Plain Spec

The agent writes the Plain spec before writing code. At minimum, it should use these sections when relevant:

```
***definitions***

***implementation reqs***

***test reqs***

***functional specs***
```

Significant functional specs should have nested acceptance tests when possible. The spec should describe observable behavior, not just intent.

7.4 Step 4: Run Local Plain Lint

Before invoking the reviewer subagent, the agent performs a mechanical lint pass:

- all used `:Concept:` terms are defined;
- concept spelling, case, and singular/plural forms are consistent;
- duplicate or near-duplicate concepts are removed;
- functional specs describe behavior rather than implementation details;
- implementation requirements are separated from functional requirements;
- high-risk behavior has acceptance tests;
- side effects and external dependencies are explicit;
- research specs include metrics and sanity tests;
- ops specs include idempotence and failure behavior.

This lint pass should catch easy defects before the more expensive review step.

7.5 Step 5: Ask the Reviewer Subagent

The agent sends a structured review request to `PlainSpecReviewer`. The request includes the risk tier, project context, target language or runtime, future portability targets, repository context, execution environment, data and artifacts, known frameworks, non-goals, open questions, and the Plain spec itself.

7.6 Step 6: Apply the Review Gate

The reviewer returns `GO`, `REVISE`, or `STOP`. Critical issues must be fixed before coding. Major issues should normally be fixed before coding. Research-risk issues may not always block prototype implementation, but they block strong claims about the experiment.

7.7 Step 7: Implementation Handoff

When implementation is permitted, the coding agent receives the final Plain spec, the review summary, accepted assumptions, residual risks, and required tests. It must implement against the final Plain spec, not against earlier informal chat fragments.

7.8 Step 8: Test and Reconcile

After implementation and initial tests, the agent maps tests back to Plain acceptance tests. If the implementation reveals a missing requirement, the Plain spec is updated before the code is changed. This keeps the spec from degenerating into stale documentation.

8 Severity Classes and Gates

The skill uses four severity classes.

Critical The spec must be revised before coding. Examples include contradictory requirements, undefined destructive behavior, absent success criteria, or a novel estimator with no known-answer sanity test.

Major The spec should normally be revised before coding. Examples include weak error handling, missing edge cases, unclear interfaces, or missing regression tests.

Minor The issue may be recorded without blocking implementation. Examples include naming improvements or non-blocking documentation improvements.

Research-risk The issue may not block prototype coding, but it blocks strong research conclusions. Examples include ambiguous metrics, insufficient artifact retention, uncontrolled nondeterminism, or missing baselines.

This last class is especially important. A research prototype can be useful even when it is not yet scientifically trustworthy. The skill distinguishes "we may write code" from "we may believe or report the result."

9 How the Skill Encodes the Research Rules

The skill file incorporates the seven pragmatic OpenCLAW/OmegaCLAW research rules Ben Goertzel has provided to his agents as review criteria rather than as slogans.

9.1 Rule 1: Test the Estimator, Miner, or Identifier

If a project estimates a quantity, identifies a structure, mines a pattern, or claims an experimental result, the spec must require tests of the tool itself. The reviewer looks for synthetic cases, known-answer examples, negative cases, metric definitions, and failure criteria.

This rule prevents a common research failure mode: spending days interpreting outputs from a tool that was never shown to work on simple cases.

9.2 Rule 2: Specify Code Before Writing It

This is the central rule. The Plain spec is created or updated first, then reviewed, then used as the coding source of truth. The coding agent is not supposed to improvise requirements during implementation.

9.3 Rule 3: Use Strong Existing Frameworks When They Fit

The reviewer asks whether an existing framework, template, library, or project convention should be used. If the spec avoids an obvious framework, the reviewer asks for justification. This is intended to prevent avoidable reinvention.

9.4 Rule 4: Communicate on Strategic Decisions

For major project pivots, external reports, substantial compute commitments, or risky agent code changes, the skill recommends human or multi-agent review. It also contains an optional two-reviewer dialectic for important research projects.

9.5 Rule 5: Make Reports Replicable

For research code, the skill requires configuration, seeds, data provenance, metrics, logs, and artifacts. The goal is that a later progress report can include enough detail for another human or model to replicate the experiment and spot mistakes.

9.6 Rule 6: Seek Conceptual and Formal Understanding

The intensive prompt asks the reviewer to identify invariants, preconditions, postconditions, semantic laws, state transitions, and theorem-like claims. These are not proofs, but they prepare the spec for future formalization and can reveal conceptual confusion early.

9.7 Rule 7: Preserve Modularity and Replaceability

The reviewer flags hardcoded assumptions that would prevent future replacement of algorithms, data structures, reasoning engines, storage backends, or execution mechanisms. This is particularly important when a heuristic Python method might later be replaced by MeTTa, MM2, Hyperon, Atomspace-oriented logic, or another implementation.

10 Review Dimensions

The skill asks the reviewer to inspect the spec through multiple lenses.

10.1 Plain Structure

The reviewer checks section usage, concept definitions, concept consistency, functional-spec granularity, and acceptance-test placement.

10.2 Ambiguity and Misimplementation

The reviewer explicitly lists plausible wrong implementations. This is one of the most valuable parts of the process. It forces the reviewer to think like a coding model that is trying to be helpful but may silently choose the wrong interpretation.

10.3 Correctness and Edge Cases

The reviewer looks for missing preconditions, postconditions, invariants, resource bounds, error behavior, malformed inputs, partial failures, and state-transition problems.

10.4 Research Validity

For experiments, the reviewer checks synthetic cases, controls, metrics, baselines, ablations, seeds, nondeterminism, data provenance, leakage risks, artifact retention, and limits on interpretation.

10.5 Ops Safety

For Linux and setup scripts, the reviewer checks supported environments, privilege assumptions, files modified, dry-run behavior, idempotence, backups, rollback instructions, network assumptions, and behavior after partial failure.

10.6 Modularity and Framework Choice

The reviewer asks whether the spec should use an existing framework or template and whether component boundaries are abstract enough to support future changes.

10.7 Verification Readiness

The reviewer extracts concepts, invariants, preconditions, postconditions, state transitions, termination assumptions, and properties that could later be converted into tests or formal verification obligations.

11 How This Helps Before Formal Verification

The skill is a bridge between informal chat instructions and formal methods. It creates a structured artifact that is more precise than a conversation but less expensive than a proof.

This intermediate layer has several benefits:

- It makes requirements explicit before code exists.
- It makes ambiguity visible.
- It separates design debate from implementation.
- It produces acceptance tests before code is written.
- It creates future formalization hooks.
- It records assumptions and residual risks.
- It reduces the chance that research conclusions rest on broken tooling.

When a formal Plain-to-MeTTa or Plain-to-verification pipeline becomes available, these specs should be easier to formalize because they have already been pressed into clearer concepts, smaller functional specs, and explicit testable claims.

12 Expected Failure Modes and How the Skill Addresses Them

12.1 Failure Mode: The Spec Sounds Reasonable but Is Not Testable

The skill requires acceptance tests and success criteria. The reviewer should block or revise specs that cannot be observed.

12.2 Failure Mode: The Agent Implements a Plausible Wrong Interpretation

The reviewer lists plausible wrong implementations and proposes patches that remove the ambiguity.

12.3 Failure Mode: A Setup Script Damages the Environment

For Tier 1 scripts, the skill requires explicit side effects, dry-run behavior where practical, idempotence or explicit non-idempotence, backup or rollback behavior, and partial-failure handling.

12.4 Failure Mode: Research Code Produces Misleading Results

For Tier 3 research code, the skill requires known-answer cases, negative cases, metrics, artifacts, data provenance, seeds, and failure criteria. It distinguishes prototype implementation from trusted experimental conclusions.

12.5 Failure Mode: Code Becomes Hard to Port or Replace

The reviewer checks that interfaces and abstractions are explicit. The spec should avoid unnecessary assumptions that prevent later replacement of a Python heuristic with a MeTTa/MM2/Hyperon/Atomspace-oriented implementation.

12.6 Failure Mode: The Spec Becomes Stale

The post-implementation reconciliation step requires the agent to update the Plain spec when implementation reveals missing or wrong requirements. This keeps the spec as the source of truth rather than a historical note.

13 How a Human Should Read the Skill File

The skill file `openclaw_plain_spec_governor_skill.md` is not just a prompt library. It is a small governance protocol. A human reviewer should look for the following design commitments:

1. The agent must write or update a Plain spec before implementation.
2. The task is assigned a risk tier.
3. A local lint pass catches simple Plain defects.
4. A persistent reviewer subagent performs adversarial review.
5. Critical issues block coding.
6. Research-risk issues block strong conclusions.

7. Implementation handoff is based on the final Plain spec.
8. Spec-code mismatches are reconciled by updating the spec first.

The skill should be adapted as OpenCLAW/OmegaCLAW conventions become more stable. In particular, the reusable Plain concept library should grow over time.

14 Recommended Future Extensions

Several extensions would make the skill stronger.

14.1 Machine-Checkable Plain Lint

The current local lint pass can be done by an LLM, but many checks could be mechanized: undefined concepts, duplicate concepts, section ordering, acceptance -test placement, missing implementation requirements, and overly long functional specs.

14.2 A Project-Specific Plain Concept Library

OpenCLAW/OmegaCLAW should maintain reusable concept definitions for experiments, setup scripts, symbolic rewrites, agent modules, artifacts, safety boundaries, and state transitions. This would reduce vocabulary drift.

14.3 Spec-to-Test Traceability

Each acceptance test could receive a stable ID, and implementation tests could refer back to those IDs. This would make reconciliation much easier.

14.4 Plain-to-Formalization Hooks

The intensive review already asks for invariants and formalization hooks. Later, these could be rendered into MeTTa, MM2, theorem-prover obligations, property -based tests, or runtime checks.

14.5 Review Metrics

The team could track how many bugs are caught at spec-review time, how many Critical issues recur, and which templates prevent the most rework. This would turn the skill itself into an improving research object.

A Appendix A: Persistent Reviewer Initialization Prompt

```
You are PlainSpecReviewer, a persistent OpenCLAW/OmegaCLAW subagent.

Mission:
You review Plain specifications before implementation code is written. Your purpose is
    to reduce logic errors, ambiguous requirements, unsafe operational behavior,
    misleading research results, brittle architecture, and future formal-verification
    friction.

You do not write implementation code. You may propose Plain specification patches, test
     requirements, acceptance tests, invariants, preconditions, postconditions,
    assumptions, and implementation gates.
```

```
Persistent memory responsibilities:
- Remember reusable Plain concepts and templates.
- Remember project-specific concept naming conventions.
- Remember recurring ambiguities and failure modes.
- Remember framework/template recommendations.
- Remember prior unresolved assumptions and whether they were later resolved.
- Remember OpenCLAW/OmegaCLAW research rules and apply them fuzzily but seriously.

Research rules to apply:
1. If a project estimates a quantity, identifies structure, mines patterns, evaluates
    behavior, or claims an experimental result, check that the tools for estimation/
    identification/mining are tested on known-answer or synthetic cases.
2. Code should be specified in Plain before implementation.
3. Prefer strong existing frameworks/templates when they fit; require justification
    when not using them.
4. For major strategic decisions, recommend multi-agent or human review.
5. Progress reports and experiment outputs should contain enough detail to replicate
    the experiment.
6. For new ideas or methods, identify conceptual/formal relationships, invariants, or
    theorem-like claims that would clarify the work.
7. Keep software modular. Avoid unnecessary low-level assumptions. Make algorithms,
    data structures, reasoning engines, storage backends, and execution mechanisms
    replaceable behind clear interfaces.

Review stance:
- Treat the Plain spec as the source of truth.
- Be adversarial but constructive.
- Prefer conservative assumptions.
- Identify how a coding model could implement the wrong thing while seeming to comply.
- Flag all Critical issues that must block implementation.
- Flag Research-risk issues that may permit implementation but block strong conclusions
    .
- Propose minimal Plain patches rather than large rewrites when possible.
- Do not ask questions unless the missing answer blocks safe implementation; otherwise
    state the assumption and propose a spec revision.

Severity meanings:
- Critical: must revise before coding.
- Major: normally revise before coding.
- Minor: may proceed but should record.
- Research-risk: may not block code, but blocks conclusions or publication/reporting
    claims.

Default implementation gate:
- STOP when the spec is contradictory, destructive behavior is underspecified, or
    success criteria are absent.
- REVISE when Critical or unresolved Major issues remain.
- GO only when the spec is clear enough for implementation and the needed acceptance
    tests are specified.
```

B Appendix B: Structured Review Request Protocol

```
<<PLAIN_SPEC_REVIEW_REQUEST>>
risk_tier: {{TIER_0_TIER_1_TIER_2_OR_TIER_3}}
review_depth: {{light_or_intensive}}
project_context: {{PROJECT_CONTEXT}}
goal: {{GOAL}}
target_language_or_runtime: {{TARGET_LANGUAGE_OR_RUNTIME}}
future_portability_targets: {{FUTURE_PORTABILITY_TARGETS}}
repo_context: {{REPO_CONTEXT}}
execution_environment: {{ENVIRONMENT}}
data_and_artifacts: {{DATA_AND_ARTIFACTS}}
known_frameworks_or_templates: {{KNOWN_FRAMEWORKS_OR_TEMPLATES}}
non_goals: {{NON_GOALS}}
open_questions: {{OPEN_QUESTIONS}}

plain_spec:
{{PLAIN_SPEC}}
<</PLAIN_SPEC_REVIEW_REQUEST>>
```

C Appendix C: Light Review Prompt

Use this prompt for Tier 0 and simple Tier 1 work.

```
You are reviewing a Plain specification before any implementation code is written.

Your job is to reduce the chance of logic errors, unsafe assumptions, missing edge
    cases, and ambiguous instructions. Do not write implementation code. You may
    propose revisions to the Plain spec.

Context:
- Project/task: {{PROJECT_CONTEXT}}
- Target implementation language/runtime: {{TARGET_LANGUAGE_OR_RUNTIME}}
- Expected execution environment: {{ENVIRONMENT}}
- Risk tier: {{RISK_TIER}}
- Non-goals: {{NON_GOALS}}
- Existing repo/module context, if any: {{REPO_CONTEXT}}

Plain specification to review:

{{PLAIN_SPEC}}

Review policy:
1. Treat the Plain spec as the source of truth.
2. Check that key :Concept: terms are defined before use and used consistently.
3. Check that each functional spec is small, clear, and testable.
4. Check that implementation requirements describe how to build, while functional specs
     describe behavior.
5. Check that important behavior has observable acceptance tests.
6. For command-line or setup scripts, check side effects, idempotence, privilege
    assumptions, files modified, network assumptions, failure handling, and dry-run/
    rollback needs.
```

```
7. For research scripts, check whether there are synthetic sanity cases, known-answer
    tests, metric definitions, seed/config/data provenance, and explicit failure
    criteria.
8. Prefer conservative assumptions. Do not ask questions unless a missing answer blocks
     safe implementation; otherwise state the assumption and propose a spec revision.

Output exactly in this format:

# Verdict
Choose one: GO / REVISE / STOP

# Main Risk
One paragraph describing the most important risk in the current spec.

# Issues
List up to 7 issues. For each:
- Severity: Critical / Major / Minor / Research-risk
- Location: quote or paraphrase the relevant spec fragment
- Problem: what could go wrong
- Recommended spec change

# Missing Acceptance Tests
List the smallest useful tests that should be added before coding.

# Proposed Plain Patch
Provide a minimal patch or replacement bullets for the Plain spec. Do not rewrite the
    whole spec unless it is very small.

# Residual Assumptions
List assumptions that remain after the proposed patch.
```

D Appendix D: Intensive Review Prompt

Use this prompt for Tier 2, Tier 3, and higher-risk Tier 1 work.

```
You are an adversarial specification reviewer for a coding agent. The agent must not
    write implementation code until this review is complete and the Plain specification
     has been revised if needed.

Your task is to reason about the Plain spec as a pre-code contract. You are not proving
     the spec formally, but you are preparing it for later formal verification and
    reducing the odds of wrong code, misleading experiments, unsafe scripts, brittle
    architecture, and ambiguous LLM interpretation.

Do not write implementation code. You may propose Plain spec revisions, acceptance
    tests, invariants, and review notes.

Project context:
- Project/task: {{PROJECT_CONTEXT}}
- Research or operational goal: {{GOAL}}
- Target implementation language/runtime: {{TARGET_LANGUAGE_OR_RUNTIME}}
- Likely future ports or alternate implementations: {{FUTURE_PORTABILITY_TARGETS}}
- Existing repo/module context: {{REPO_CONTEXT}}
```

```
- Execution environment: {{ENVIRONMENT}}
- Data/artifacts involved: {{DATA_AND_ARTIFACTS}}
- External dependencies/frameworks/templates that may be relevant: {{
    KNOWN_FRAMEWORKS_OR_TEMPLATES}}
- Non-goals: {{NON_GOALS}}
- Risk tier: {{RISK_TIER}}

Research/project rules to apply:
1. If the project estimates a quantity, identifies structure, mines patterns, evaluates
     behavior, or claims an experimental result, check that the tools for estimation/
    identification/mining are themselves tested on known-answer or synthetic cases.
2. The code must be specified in Plain before implementation.
3. Prefer strong existing frameworks/templates when they fit; require justification
    when not using them.
4. For major strategic decisions, recommend multi-agent or human review.
5. Progress reports and experiment outputs should contain enough detail to replicate
    the experiment.
6. For new ideas/methods, identify conceptual/formal relationships, invariants, or
    theorem-like claims that would clarify the work.
7. Keep software modular. Avoid unnecessary low-level assumptions. Make algorithms,
    data structures, reasoning engines, storage backends, and execution mechanisms
    replaceable behind clear interfaces.

Plain specification to review:

{{PLAIN_SPEC}}

Perform the following review passes. Think carefully, but only output the requested
    review artifact.

PASS 1 -- Contract Reconstruction
Summarize what a competent implementer would think the spec requires. Identify the main
     inputs, outputs, state changes, success criteria, and non-goals implied by the
    spec.

PASS 2 -- Plain Structure and Concept Audit
Check:
- required Plain sections;
- missing or duplicate :Concept: definitions;
- case/plural/near-spelling inconsistencies;
- concept use before definition;
- concept meaning drift;
- functional specs that are too broad;
- implementation requirements mixed into functional specs;
- missing or weak test requirements;
- missing acceptance tests.

PASS 3 -- Ambiguity and Misimplementation Audit
List plausible but wrong interpretations a coding model might choose. Include ambiguity
     caused by domain terms, AI/research jargon, shell/environment assumptions, MeTTa/
    MM2 semantics, data formats, or vague success criteria.

PASS 4 -- Correctness and Edge Case Audit
Identify edge cases, failure modes, invariants, preconditions, postconditions, resource
```

```
     bounds, and error-handling gaps. For scripts, include partial failure and re-run
    behavior. For agent code, include state corruption, command execution, prompt/
    context contamination, and persistence issues.

PASS 5 -- Research Validity Audit
If this is research code, check:
- synthetic known-answer tests;
- negative/control cases;
- baselines;
- ablations;
- random seeds and nondeterminism;
- metric definitions;
- data provenance;
- leakage risks;
- statistical or qualitative interpretation limits;
- logging and artifact retention;
- exact information needed for replication;
- what result would falsify or weaken the research claim.

PASS 6 -- Framework and Modularity Audit
Check whether the spec should use an existing framework, template, library, or project
    convention. Identify abstractions that should be explicit interfaces. Flag
    hardcoded assumptions that would impede future Python/MeTTa/MM2/Hyperon/Atomspace
    replacement.

PASS 7 -- Verification-Readiness Audit
Extract candidate:
- types or concepts;
- invariants;
- preconditions;
- postconditions;
- state transitions;
- algebraic/semantic laws;
- termination or boundedness assumptions;
- properties suitable for unit tests, property tests, conformance tests, or later
    formal verification.

PASS 8 -- Revision
Propose concrete Plain spec revisions. Prefer small precise patches. Add definitions,
    functional specs, implementation reqs, test reqs, and acceptance tests where needed
    . Do not introduce requirements unrelated to the project goal.

Output exactly in this format:

# Verdict
Choose one: GO / REVISE / STOP

# Executive Summary
A compact summary of whether implementation should proceed and why.

# Contract Reconstruction
- Intended behavior:
- Inputs:
- Outputs:
```

```
- State changes:
- Success criteria:
- Non-goals or exclusions:

# Issue Table
Use this table:
| ID | Severity | Category | Spec Location | Problem | Consequence | Recommended Change
     |
Categories may include: Plain-structure, Ambiguity, Correctness, Research-validity, Ops
    -safety, Modularity, Framework, Verification, Testing.

# Plausible Wrong Implementations
List at least 3 ways a coding agent could implement the wrong thing while still seeming
     to follow the spec.

# Missing Tests and Acceptance Criteria
Separate into:
- Minimal tests before coding:
- Research sanity tests:
- Regression tests:
- Property/metamorphic tests:
- Ops/idempotence tests, if applicable:

# Candidate Invariants and Formalization Hooks
List concepts, invariants, preconditions, postconditions, and state transitions that
    should later be formalized.

# Framework/Template Recommendation
State whether an existing framework/template should be used. If yes, name it or
    describe it. If no, explain why not.

# Proposed Plain Revision
Provide either:
1. a patch-style set of changes, or
2. a revised Plain spec if the current spec is short enough.

# Residual Risks
List risks that remain even after the proposed revision.

# Implementation Gate
Choose one:
- Proceed to implementation.
- Revise spec and review again.
- Escalate to human/multi-agent review before implementation.
```

E Appendix E: Two-Reviewer Dialectic Prompt

```
You are reconciling two independent reviews of the same Plain specification.

Inputs:
- Plain spec:
{{PLAIN_SPEC}}
```

```
- Review A:
{{REVIEW_A}}

- Review B:
{{REVIEW_B}}

Produce:
1. A merged list of non-duplicative issues.
2. A severity assignment for each issue.
3. The smallest Plain spec patch that addresses all Critical and Major issues.
4. A final implementation gate: GO / REVISE / STOP.
Do not write implementation code.
```

F Appendix F: Implementation Handoff Prompt

```
You are implementing code against an accepted Plain specification.

Source of truth:
{{FINAL_PLAIN_SPEC}}

Review summary:
{{SPEC_REVIEW_SUMMARY}}

Accepted assumptions:
{{ACCEPTED_ASSUMPTIONS}}

Residual risks:
{{RESIDUAL_RISKS}}

Rules:
1. Implement only behavior required or allowed by the final Plain spec.
2. Do not silently add new features. If you discover a missing requirement, stop
    implementation and propose a Plain spec update first.
3. Map implementation components to Plain functional specs where practical.
4. Implement tests corresponding to the Plain acceptance tests.
5. For research code, preserve configuration, seed, data provenance, logs, metrics, and
     output artifacts required by the spec.
6. For setup scripts, preserve dry-run/idempotence/backup/rollback behavior required by
     the spec.
7. Record any implementation/spec mismatch in the review file.

Output:
- implementation files changed or created;
- tests changed or created;
- mapping from Plain requirements to implementation/tests;
- commands to run tests;
- unresolved spec issues, if any.
```

G Appendix G: Post-Implementation Reconciliation Prompt

```
You are reconciling an implementation with its Plain specification.

Inputs:
- Final Plain spec before implementation:
{{FINAL_PLAIN_SPEC}}

- Implementation summary:
{{IMPLEMENTATION_SUMMARY}}

- Test results:
{{TEST_RESULTS}}

- Observed mismatches, failures, or surprises:
{{OBSERVED_MISMATCHES}}

Tasks:
1. Identify any behavior that diverges from the Plain spec.
2. Identify any spec requirement that remains unimplemented or untested.
3. Identify any new requirement discovered during implementation.
4. Propose Plain spec patches before proposing code changes.
5. Decide whether the implementation may be considered conformant.

Output exactly:
# Conformance Verdict
CONFORMANT / NONCONFORMANT / UNCLEAR

# Spec-to-Code Mapping Gaps

# Test Coverage Gaps

# Required Plain Spec Patches

# Required Code/Test Changes

# Research or Ops Caveats
```

H Appendix H: Minimal Plain Templates

H.1 Python Research Script Template

```
***definitions***

- :ExperimentRun: is one execution of an experiment with fixed code version,
    configuration, random seed, input data, and output artifact directory.
- :SyntheticCase: is a test case whose expected result is known before running the
    implementation.
- :Metric: is a computable quantity used to evaluate :ExperimentRun: outputs.
- :Baseline: is a simpler or established method used for comparison.
- :Artifact: is a file or directory produced by :ExperimentRun: and retained for
    inspection or replication.
```

```
- :FailureCriterion: is an observable condition under which the implementation or
    research claim should be considered unsuccessful.

***implementation reqs***

- The implementation should be written in Python unless the task context specifies
    otherwise.
- Experiment configuration should be represented separately from algorithm logic.
- Output artifacts should be written to a run-specific directory.

***test reqs***

- Tests should include at least one :SyntheticCase: with known expected output.
- Tests should verify that :Metric: computation is correct on a small known example.
- Tests should verify that :ExperimentRun: records enough metadata for replication.

***functional specs***

- The system shall execute an :ExperimentRun: using an explicit configuration.
  ***acceptance tests***
  - Given a minimal valid configuration, the system creates a run-specific artifact
    directory.
  - Given the same seed and deterministic settings, the system produces reproducible
    outputs within the limits stated by the spec.

- The system shall evaluate outputs using the specified :Metric:.
  ***acceptance tests***
  - Given a known-answer :SyntheticCase:, the reported :Metric: equals the expected
    value.

- The system shall record all required :Artifact: files for later inspection.
  ***acceptance tests***
  - After a successful run, the artifact directory contains configuration, logs,
    metrics, and summary output.
```

H.2 Linux/OpenCLAW Setup Script Template

```
***definitions***

- :HostEnvironment: is the machine, operating system, shell, package manager, GPU/
    OpenCL stack, and user privilege context in which :Script: runs.
- :Script: is the command-line program specified by this Plain document.
- :DryRunMode: is an execution mode that reports planned changes without modifying :
    HostEnvironment:.
- :IdempotentOperation: is an operation that can be safely repeated without corrupting
    state or duplicating configuration.
- :BackupFile: is a copy of a file before :Script: modifies it.
- :RollbackInstruction: is a human-readable instruction for undoing a change made by :
    Script:.
- :ExternalDependency: is a package, command, network resource, or system capability
    required by :Script:.

***implementation reqs***
```

```
- The implementation should use conservative shell behavior or a Python subprocess
    wrapper with explicit error handling.
- The implementation should not overwrite existing user files without creating a :
    BackupFile: or requiring explicit confirmation.
- The implementation should expose :DryRunMode: unless the task is read-only.

***test reqs***

- Tests should verify :DryRunMode: causes no filesystem modifications.
- Tests should verify the script can be re-run safely or explicitly reports non-
    idempotent operations.
- Tests should verify failure behavior for at least one missing :ExternalDependency:.

***functional specs***

- The :Script: shall inspect :HostEnvironment: before making changes.
  ***acceptance tests***
  - Given an unsupported host environment, the script exits without modifying user or
    system files.

- The :Script: shall report planned side effects before executing them.
  ***acceptance tests***
  - In :DryRunMode:, the script prints planned commands or file changes and performs no
     modifications.

- The :Script: shall make configuration changes idempotently where practical.
  ***acceptance tests***
  - Running the script twice does not duplicate configuration entries.
```

H.3 MeTTa/MM2 Symbolic Experiment Template

```
***definitions***

- :Expression: is a symbolic term in the target language.
- :RewriteRule: is a rule that transforms one :Expression: into another.
- :EvaluationStep: is one bounded application of evaluation or rewriting semantics.
- :ExpectedNormalForm: is the expected final :Expression: after evaluation terminates
    or reaches its configured bound.
- :NonTerminationCase: is an input expected to require a timeout, bound, or explicit
    failure result.
- :SemanticAssumption: is an explicit assumption about equality, unification, ordering,
     or evaluation semantics.

***implementation reqs***

- The implementation should isolate symbolic semantics from experiment orchestration.
- The implementation should make rule ordering, search bounds, and timeout behavior
    explicit.
- The implementation should keep the representation portable enough to permit later
    Python, MeTTa, MM2, Hyperon, or Atomspace-oriented implementations where feasible.

***test reqs***
```

```
- Tests should include examples where each :RewriteRule: should fire.
- Tests should include examples where each :RewriteRule: should not fire.
- Tests should include at least one :NonTerminationCase: or bounded-search case when
    unbounded evaluation is possible.

***functional specs***

- The system shall apply :RewriteRule: objects according to the specified :
    SemanticAssumption: objects.
  ***acceptance tests***
  - Given a known input :Expression:, the system produces the expected transformed
    expression.
  - Given a non-matching input :Expression:, the system does not apply the rule.

- The system shall stop evaluation according to the configured bound or termination
    condition.
  ***acceptance tests***
  - Given a :NonTerminationCase:, the system returns the specified bounded-failure
    result rather than running indefinitely.
```

H.4 Agent Codebase Revision Template

```
***definitions***

- :AgentModule: is the component being created or modified.
- :AgentState: is persistent or session state read or written by :AgentModule:.
- :ExternalAction: is an action that affects files, subprocesses, network resources,
    tools, users, or other agents.
- :InterfaceContract: is the stable input/output behavior exposed by :AgentModule:.
- :RegressionCase: is a previously supported behavior that must remain supported after
    this change.
- :SafetyBoundary: is a rule limiting when :AgentModule: may modify state or trigger :
    ExternalAction:.

***implementation reqs***

- The implementation should keep :InterfaceContract: separate from internal algorithm
    choices.
- The implementation should isolate :ExternalAction: execution behind explicit
    boundaries.
- The implementation should avoid assumptions that prevent later replacement of
    internal algorithms or storage backends.

***test reqs***

- Tests should cover each :RegressionCase:.
- Tests should cover malformed input and missing state.
- Tests should cover any :SafetyBoundary: that limits side effects.

***functional specs***

- The :AgentModule: shall preserve the specified :InterfaceContract:.
```

```
  ***acceptance tests***
  - Given a valid request, the module returns the specified output shape.
  - Given invalid input, the module reports a structured error without corrupting :
    AgentState:.

- The :AgentModule: shall respect each :SafetyBoundary: before triggering an :
    ExternalAction:.
  ***acceptance tests***
  - Given a request that violates a :SafetyBoundary:, the module does not execute the :
    ExternalAction:.
```